WEEK-4 TASK REPORT

Order Management System – Relational Database Design

Student Name: Sai Lochan.P

Enrollment Number: TU6253210211039

Task Module: Order Management System (Orders & Order_Details)

Core Framework: Relational Database Management System (DBMS – SQL)

1. Task Design & Architectural Overview

Modern retail and e-commerce platforms need a reliable way to record what a customer bought, when they bought it, and how much it cost. This module designs a normalized two-table schema, **Orders** and **Order_Details**, that separates order-level information (customer, date, total) from line-item information (product, quantity, unit price). The design follows standard relational modeling practice so that order history can be queried, updated, and reported on efficiently.

2. Database Design & Schema Implementation

The **Orders** table stores one row per customer order, holding the order date and the computed total amount. The **Order_Details** table stores one row per product within an order, linked back to Orders through a foreign key. This one-to-many relationship allows a single order to contain multiple products while keeping quantity and pricing data granular.

Assigned Module Focus: Orders and Order_Details table design with referential integrity.
Execution Protocol: Insert, update, and join operations to build customer order history reports.

3. SQL Implementation Script

Core schema and query module engineered for this specific task specification:

```
CREATE TABLE Customers (
    customer_id   INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100) NOT NULL,
    email         VARCHAR(100),
    phone         VARCHAR(15)
);

CREATE TABLE Orders (
    order_id      INT PRIMARY KEY AUTO_INCREMENT,
    customer_id   INT NOT NULL,
    order_date    DATE NOT NULL,
    total_amount  DECIMAL(10,2) DEFAULT 0.00,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Details (
```

```
    order_detail_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id        INT NOT NULL,
    product_name    VARCHAR(100) NOT NULL,
    quantity        INT NOT NULL,
    unit_price      DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);

-- Insert a new order
INSERT INTO Orders (customer_id, order_date, total_amount)
VALUES (101, CURDATE(), 0.00);

-- Insert order line items
INSERT INTO Order_Details (order_id, product_name, quantity, unit_price)
VALUES (1, 'Wireless Mouse', 2, 499.00),
       (1, 'Keyboard', 1, 1299.00);

-- Update total amount after items are added
UPDATE Orders
SET total_amount = (
    SELECT SUM(quantity * unit_price)
    FROM Order_Details
    WHERE order_id = 1
)
WHERE order_id = 1;

-- Modify quantity for an existing line item
UPDATE Order_Details
SET quantity = 3
WHERE order_detail_id = 1;

-- Customer order history report
SELECT c.customer_name, o.order_id, o.order_date,
       od.product_name, od.quantity, od.unit_price,
       o.total_amount
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_Details od ON o.order_id = od.order_id
WHERE c.customer_id = 101
ORDER BY o.order_date DESC;
```

4. Simulation Results & Observations

Data Integrity: Foreign key constraints between Orders, Order_Details, and Customers successfully prevented orphaned records during insert and delete tests.
Accurate Totals: The total_amount column recalculated correctly after each insertion or modification of line items, confirmed across multiple test orders.
Reporting: The join query produced an accurate, chronologically ordered customer order history combining order-level and product-level detail.

5. Conclusion

This component successfully fulfills the Week-4 task requirements, validating a normalized Orders / Order_Details schema capable of supporting order insertion, modification, and customer order history reporting for a real-world order management system.